

NL2SQL 프로젝트 최종 보고서

프로젝트: 학회 관리 DB 대상 한국어 NL2SQL 시스템
최종 버전: v46
최종 성능: Blind 86.4% (TYPE_A 보정) / Targeted 87.3%
작성일: 2026-06-12

1. 연구 목적

학회 관리 시스템 DB에 저장된 데이터를 비개발자도 자연어로 조회할 수 있도록,
한국어 질문 → **MySQL 쿼리 자동 변환** 모델을 개발한다.

입력: "2026년 춘계 학술대회에 등록한 정회원 수는?"
출력:

```
SELECT COUNT(*) AS cnt
FROM sc_conf_registrant r
JOIN sc_conf_conference c ON r.conf_id = c.id
WHERE c.season = 'spring' AND c.conferenceStartDate LIKE '2026%'
AND r.fee_code IN (SELECT code FROM sc_conf_fee WHERE annual_fee = 1)
LIMIT 100;
```

기존 방법의 한계

방법	문제
범용 LLM (GPT-4o 등)	학회 특화 스키마 모름, 도메인 규칙(승인/회비 만료 등) 오해, API 비용+데이터 외부 전송
직접 SQL 작성	비개발자 진입 장벽 높음
규칙 기반 파싱	표현 다양성 대응 불가

해결 접근

SQL 특화 사전학습 LLM
+ QLoRA 도메인 파인튜닝 (학회 DB 특화 SQL 페어)
+ 추론 시점 기술 (Schema Pruning, RAG, Postprocessing, Validator)
→ 온프레미스에서 동작하는 도메인 특화 NL2SQL

2. 모델 및 연구 환경

2.1 베이스 모델

항목	내용
모델	defog/sqlcoder-7b-2

항목	내용
파라미터	7B (LLaMA-2 아키텍처 기반)
특징	SQL DDL, 쿼리, DB 스키마 텍스트로 대규모 사전 학습된 SQL 특화 오픈소스 모델
선택 이유	SQL 문법 기본 탑재 → 소량 도메인 데이터로도 높은 정확도 달성 가능

범용 LLM 대신 SQL 특화 모델을 선택한 이유:

- **비용**: 상업 API 없이 온프레미스 운용 가능
- **보안**: 민감한 회원 데이터가 외부로 전송되지 않음
- **효율**: SQL 문법을 이미 알고 있으므로 도메인 지식만 추가 학습하면 됨

2.2 파인튜닝 방법: QLoRA

7B 모델을 full fine-tuning하려면 옵티마이저 포함 84GB 이상의 VRAM이 필요하다.

QLoRA (Quantized Low-Rank Adaptation)로 이를 해결했다.

전통적 파인튜닝: W (7B 파라미터 전체 업데이트) → VRAM 84GB+ 필요
 QLoRA: W는 4-bit NF4로 고정 (~4GB) + LoRA 어댑터(~0.3GB)만 학습
 → RTX A6000 단일 GPU로 훈련 가능

LoRA 원리: 원본 가중치 행렬 W 대신 저랭크 분해 $\Delta W = A \times B$ 학습.

W가 4096×4096이면 full: 16.7M 파라미터, LoRA r=16: 131K 파라미터 (0.78%).

하이퍼파라미터	값	설명
<code>lora_r</code>	16	랭크. 표현력 ↔ 메모리 균형
<code>lora_alpha</code>	32	스케일 ($\alpha/r=2$, 실효 학습률 ×2)
<code>lora_dropout</code>	0.05	소량 데이터 환경 과적합 방지
<code>target_modules</code>	q,k,v,o,gate,up,down proj	Attention + FFN 전 레이어
<code>learning_rate</code>	2e-4	코사인 스케줄러
<code>batch_size</code>	4×4=16 (gradient_accumulation)	
<code>early_stopping</code>	patience=3 (val_loss)	
<code>max_length</code>	4096	최종 설정

2.3 연구 환경

항목	내용
GPU	NVIDIA RTX A6000 (47GB VRAM)
훈련 시간	버전당 약 6~7시간
프레임워크	HuggingFace Transformers + PEFT (BitsAndBytes)

항목	내용
평가 DB	MySQL (학회 예시 더미 데이터)
임베딩 모델	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 (384차원, 50개국어)

3. 대상 테이블 (6개)

3-table 시기(v1~v21): sc_users, sc_class, sc_user_orders
6-table 시기(v22~v46): 위 3개 + sc_conf_conference, sc_conf_fee, sc_conf_registrant 추가

3.1 회원 관련 테이블 (초기부터)

sc_users — 학회 회원 정보 (17 컬럼)

주요 컬럼	타입	설명
USER_NO	varchar(20)	사용자 번호 (PK)
SOCIETY_ID	int	학회 ID
class	int	회원 등급 ID (sc_class.id 참조)
approved	tinyint	승인 여부 (0: 미승인, 1: 승인)
approve_ready	tinyint	승인 준비 여부
membership_date	datetime	회비 만료일 (※ 가입일 아님)
enabled	tinyint	사용 여부 (0: 비활성, 1: 활성)
mileage	int	마일리지
reg_date	datetime	등록일시

핵심 도메인 규칙: membership_date >= CURDATE() = 유효 회원, < CURDATE() = 만료 회원

sc_class — 회원 등급 분류 (9 컬럼)

주요 컬럼	설명
id	등급 ID (sc_users.class와 JOIN)
society_id	학회 ID
name_kor	등급 한글명 (정회원, 준회원, 종신회원 등)
type	등급 유형 (BASIC, EXEMPT, LIFETIME)
use	사용 여부

sc_user_orders — 결제/주문 이력 (25 컬럼)

주요 컬럼	설명
user_no	사용자 번호 (sc_users.USER_NO와 JOIN)

주요 컬럼	설명
price	주문 금액
confirm	결제 확인 여부
refund	환불 여부
order_datetime	주문일시
confirm_datetime	결제 확인일시

3.2 학술대회 관련 테이블 (v22부터 추가)

sc_conf_conference — 학술대회 정보 (32 컬럼)

주요 컬럼	설명
id	학술대회 ID
nameKor / nameEng	대회명 (한글/영문)
season	계절 (spring, fall 등)
conferenceStartDate / conferenceEndDate	학술대회 기간
absStartDate / absEndDate	초록 제출 기간
submissionStartDate / submissionEndDate	논문 제출 기간
generalPreregStartDate / generalPreregEndDate	일반 사전등록 기간
location	개최 장소

특이사항: 컬럼명이 camelCase (snake_case 아님). DDL COMMENT로 의미 보정 필요.

sc_conf_fee — 학술대회 등록비 (15 컬럼)

주요 컬럼	설명
conf_id	학술대회 ID (sc_conf_conference와 JOIN)
code	등록비 코드
name	등록비 명칭
amount	금액
annual_fee	연회비 여부
manuscript_fee	논문 등록비 여부
start_date / end_date	등록비 유효 기간

sc_conf_registrant — 학술대회 등록자 정보 (37 컬럼)

주요 컬럼	설명
user_no	사용자 번호
conf_id	학술대회 ID
fee_code	적용 등록비 코드
institution	소속 기관
name_eng	등록자 영문명
order_confirm_date	결제 확인일
manuscript_submit_id	논문 제출 ID

3.3 테이블 간 주요 JOIN 관계

```

sc_users ——— sc_class
(class = id, SOCIETY_ID = society_id)

sc_users ——— sc_user_orders
(USER_NO = user_no)

sc_conf_conference ——— sc_conf_fee
(id = conf_id)

sc_conf_conference ——— sc_conf_registrant
(id = conf_id)

sc_conf_registrant ——— sc_users
(user_no = USER_NO) ← 6-table 교차 JOIN

```

4. 추론 파이프라인

4.1 전체 흐름

```

자연어 질문 입력
|
▼
[Step 0] IntentClassifier – 비관련 질문 차단
| "맛집 추천" → 즉시 거부 (LLM 추론 비용 없음)
| DB 관련 질문 → 계속 진행
|
▼
[Step 1] Schema Pruning – 관련 테이블 DDL만 선택
| 질문 키워드 분석 → 6개 테이블 중 관련 테이블만 추출
| Full DDL 1,529 토큰 → Pruned DDL ~368 토큰
|
▼
[Step 2] RAG – 유사 질문-SQL 예제 검색 (선택적)
| 4,412개 훈련 페어에서 코사인 유사도 top-3 검색

```

```

| 같은 도메인 그룹 내에서만 검색 (회원/학술대회/등록비 등)
▼
[Step 3] 프롬프트 구성
| 질문 + Pruned DDL (+ DDL COMMENT) + RAG 예제
| 총 입력 토큰: ~368 (pruned, RAG 있을 때)
▼
[Step 4] 모델 추론 (defog/sqlcoder-7b-2 + QLoRA 어댑터)
| max_new_tokens=400, Greedy Decoding (do_sample=False)
| [/SQL] 토큰 감지 시 즉시 중단
▼
[Step 5] postprocess_sql() - 형식 정규화
| INTERVAL 정규화, DATE_FORMAT 교정, CURRENT_DATE → CURDATE()
| Alias 표준화, GROUP BY alias 확장, MAX/MIN → ORDER BY LIMIT 1
▼
[Step 6] SQLValidator - 안전 검사 + 환각 제거
| DML 차단 (INSERT/UPDATE/DELETE/DROP 등)
| 환각 컬럼 자동 제거 (스키마에 없는 WHERE 조건 삭제)
| LIMIT 100 자동 추가
▼
최종 SQL 반환

```

4.2 DDL COMMENT

schema.sql에 각 컬럼의 도메인 규칙을 주석으로 명시해 모델에 힌트를 제공한다.

```

-- DDL COMMENT 예시 (sc_users)
membership_date DATETIME COMMENT '회비 만료일. >= CURDATE()이면 유효, < CURDATE()이면 만료',
approved          TINYINT  COMMENT '0: 미승인, 1: 승인 완료',
approve_ready     TINYINT  COMMENT '0: 준비 안 됨, 1: 승인 대기 중',

-- sc_conf_conference (camelCase 컬럼 보정)
conferenceStartDate DATETIME COMMENT '학술대회 시작일',
season              VARCHAR  COMMENT '계절 구분: spring, fall, winter, summer',

```

5. 3-table 시기: v21까지의 성과

5.1 3-table 학습 여정 (v1~v21)

초기 sc_users 단일 테이블에서 시작해 sc_class, sc_user_orders를 순차적으로 추가했다.

버전	훈련 데이터	주요 사건
v1~v15	~700개	초기 탐색, sc_users 단독
v16	866개	평가 파이프라인 안정화
v17	912개	blind1 100%, blind2 98% 달성
v18	905개	blind3/4 테스트셋 추가

버전	훈련 데이터	주요 사건
v19	1,073개	sc_class/orders 패턴 첫 시도 → blind5 18.6% 실패
v20	1,248개	DDL 3-table 통일 → blind5 91.4%, 하지만 catastrophic forgetting
v21	1,414개	counter-example 추가로 전면 수정 완료

5.2 핵심 기술적 문제 해결

① blind5 전멸 (v19: 18.6%)

finetune.py와 evaluate.py DDL에 sc_class, sc_user_orders 스키마 누락.

모델이 학습 시 보지 못한 테이블에 JOIN을 생성하도록 강요됨.

→ DDL 3-table 통일로 해결.

② Catastrophic Forgetting (v20)

sc_class JOIN 예제 과다 추가 후, "1등급 회원 수" 같은 숫자 등급 쿼리에도 JOIN 자동 생성.

```
-- 기대 (JOIN 불필요)
SELECT COUNT(*) FROM sc_users WHERE class = 1;

-- v20 오류 생성
SELECT COUNT(*) FROM sc_users
JOIN sc_class ON sc_users.class = sc_class.id
WHERE sc_class.rank = 2; -- 존재하지 않는 컬럼까지 환각
```

→ 숫자 등급 → WHERE class = N (JOIN 없음) counter-example 30개 추가로 해결.

5.3 v21 최종 성과 (3-table 시기 마무리)

훈련: 1,414 SQL 페어 | train_loss=0.047, eval_loss=0.005, epoch=6.9

테스트셋	문항 수	정확 일치	유연 일치	합계
blind1 (기본 다양성)	198	75.8%	20.7%	96.5%
blind2 (고난도 패턴)	189	76.7%	18.0%	94.7%
blind3 (파라프레이즈)	68	95.6%	1.5%	97.1%
blind4 (복합 조건)	49	89.8%	8.2%	98.0%
blind5 (3-table JOIN)	70	80.0%	10.0%	90.0%

blind1~4 평균: 96.6% / blind5: 90.0%

5.4 base 모델 vs v21

모델	blind1~4 평균	blind5
순수 base (defog/sqlcoder-7b-2)	~21%	~19%

모델	blind1~4 평균	blind5
v21 (Fine-tuned)	96.6%	90.0%
격차	+75.6%p	+71%p

base 모델은 SQL 형식(SELECT, FROM, WHERE)은 알지만 학회 도메인 규칙을 전혀 모른다.
Fine-tuning이 이 gap을 메웠다.

6. 6-table 확장: v22~v46 전환

6.1 무엇이 바뀌었나

3개 테이블에서 6개 테이블로 확장하면서 복잡도가 크게 증가했다.

항목	3-table 시기	6-table 시기
테이블 수	3	6
총 컬럼 수	~51	~135
도메인	회원 관리	회원 + 학술대회 전체
Full DDL 토큰 수	~1,335	~1,529 (w/o COMMENT) / 3,635 (w/ COMMENT)
훈련 데이터	1,414개	최종 4,412개
context window 부담	낮음	COMMENT 포함 시 4,096 한계 근접

새로 추가된 도메인:

- 학술대회 일정 관리 (개회식/논문/초록/등록 각 시작~종료일)
- 계절별/연도별 학술대회 조회
- 등록비 구조 (일반/학생/연회비/논문등록비)
- 등록자-대회-등록비 3-way JOIN
- camelCase 컬럼명 처리 (conferenceStartDate 등)

6.2 새로 적용한 기술들

기존 3-table 시기에는 단순 파인튜닝만으로 충분했지만, 6-table 확장 후 여러 추론 시점 기술이 도입됐다.

기술	도입 이유
DDL COMMENT	camelCase 컬럼명 의미 보정, season 값 힌트, 날짜 컬럼 도메인 규칙
Schema Pruning	6개 테이블 Full DDL이 context window의 37~88%를 차지 → 관련 테이블만 선택
RAG	복잡한 3-way JOIN 패턴을 예제로 제시 → 모델이 패턴 참조
Postprocessing	날짜 표현 정규화, INTERVAL/DATEDIFF 통일, 불필요 ORDER BY 제거
SQL Validator	환각 컬럼 제거, DML 차단, LIMIT 자동 추가

6.3 3-layer 평가 체계 구축

6-table 환경에서 성능을 체계적으로 검증하기 위해 3계층 평가 체계를 새로 설계했다.

레이어	파일	문항 수	신뢰 구간	목적
Blind	blind_test_all6_tables.json	214	±6.1%p	전체 도메인 일반화 성능
Coverage	column_coverage_test_v2.json	680	±3.7%p	6개 테이블 컬럼별 커버리지
Targeted	targeted_test_v2.json	300	±18%p	10개 패턴 집중 검증

Targeted 10개 패턴:

패턴	내용
1	날짜 범위 (기간 조회)
2	집계 (COUNT, SUM, AVG)
3	다중 조건 AND
4	sc_class JOIN (등급명 조회)
5	sc_user_orders JOIN (결제 이력)
6	sc_conf_conference 단독
7	sc_conf_fee JOIN
8	sc_conf_registrant JOIN
9	크로스 도메인 (회원+학술대회)
10	IS NULL / IS NOT NULL 패턴

6.4 MySQL 실행 기반 TYPE_A 보정

문자열 비교만으로는 의미상 동등한 SQL을 불일치로 처리하는 허위음성 문제가 있다.

```
-- Gold SQL
SELECT id, nameKor FROM sc_conf_conference WHERE year = 2026 ORDER BY nameKor;

-- Pred SQL (ORDER BY 컬럼만 다름)
SELECT id, nameKor FROM sc_conf_conference WHERE year = 2026 ORDER BY id;

-- 문자열 비교: 불일치
-- MySQL 실행 결과: 동일 → TYPE_A (정답으로 보정)
```

유형	의미	처리
TYPE_A	Gold = Pred 실행 결과 완전 일치	정답 처리 (보정 점수에 반영)
TYPE_C	Gold ≠ Pred 실행 결과	진짜 모델 오류
EXEC_ERR	Pred SQL 실행 오류	잘못된 SQL 생성

보정 점수 = 문자열 점수 + TYPE_A 건수 → 실제 모델 성능에 가장 가까운 수치.

7. v46: base 모델 대비 성능

7.1 v46이 기준 버전인 이유

버전	Blind	Coverage	Targeted	Blind RAG	Targeted RAG
v40	87.9%	73.8%	73.3%	—	79.7%
v42	87.4%	77.4%	84.0%	81.3%	81.0%
v43	86.0%	78.5%	81.7%	84.1%	85.7%
v44	85.5%	77.9%	84.0%	86.9%	87.0%
v45	92.1%	76.9%	85.7%	89.3%	83.7%
v46	86.4%	75.9%	87.3%	83.6%	88.3%

v45는 Blind 92.1%로 최고이지만 Targeted 균형이 낮음.

v46은 Targeted 87.3% + Targeted RAG 88.3%로 실용 패턴 성능이 가장 우수.

v46 훈련 조건: 4,412 SQL 페어 | epoch ~8 | train_loss ~0.04

7.2 base 모델 vs v46 비교 (Blind 214개)

모델	정확 일치	유연 일치	합계 (raw)	TYPE_A 보정
순수 base	0.5%	21.0%	21.5%	~30%
v46	41.1%	42.1%	83.2%	86.4%
격차	+40.6%p	+21.1%p	+61.7%p	+56.4%p



해석:

- base는 SQL 구조(JOIN, 테이블 선택)는 어느 정도 파악(유연 21%)하지만
- 도메인 특화 WHERE 조건·컬럼 값(정확 0.5%)을 전혀 못 맞춘다
- Fine-tuning이 정확 일치를 **82배** (0.5% → 41.1%) 향상시켰다

8. 기술별 기여도 분석

방법론: Forward Selection — 순수 base에 기술 하나씩 독립 추가

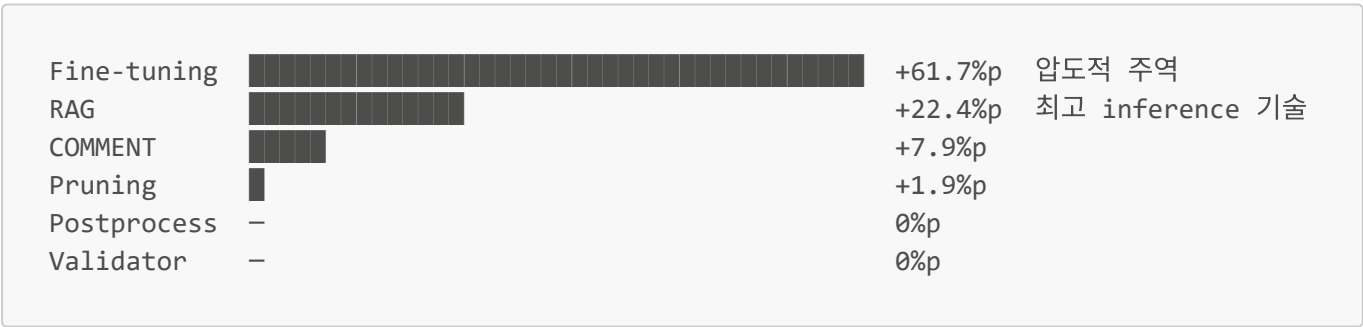
기여도(기술 X) = [base + X만 적용] - [base]

8.1 실험 설계 (7개 조건)

실험	모델	추가 기술	DDL	COMMENT	입력 토큰
C0	BASE	없음 (기준선)	Full	X	1,529
C0v	V46	Fine-tuning	Full	X	1,529
C1	BASE	DDL COMMENT	Full	✓	3,635
C2	BASE	Schema Pruning	Pruned	X	~368
C3	BASE	Postprocessing	Full	X	1,529
C4	BASE	RAG (few-shot)	Pruned	X	~368
C5	BASE	SQL Validator	Full	X	1,529

8.2 기술별 독립 기여도

순위	기술	순수 base → 적용 후	기여도
—	순수 base (C0, 기준선)	21.5%	—
1	Fine-tuning (v46)	21.5% → 83.2%	+61.7%p
2	RAG (few-shot)	21.5% → 43.9%	+22.4%p
3	DDL COMMENT	21.5% → 29.4%	+7.9%p
4	Schema Pruning	21.5% → 23.4%	+1.9%p
5	Postprocessing	21.5% → 21.5%	0%p
5	SQL Validator	21.5% → 21.5%	0%p



8.3 정확 일치 / 유연 일치 세부 수치

실험	정확 일치	유연 일치	합계	불일치
C0: 순수 base	0.5%	21.0%	21.5%	168/214
C0v: Fine-tuning	41.1%	42.1%	83.2%	36/214
C1: + COMMENT	0.5%	29.0%	29.4%	151/214
C2: + Pruning	0.5%	22.9%	23.4%	164/214
C3: + Postprocess	0.5%	21.0%	21.5%	168/214

실험	정확 일치	유연 일치	합계	불일치
C4: + RAG	13.6%	30.4%	43.9%	120/214
C5: + Validator	0.5%	21.0%	21.5%	168/214

8.4 핵심 발견

발견 1 — RAG의 기여는 few-shot 예제가 주도한다

RAG = Schema Pruning(테이블 선택) + few-shot 예제 2요소의 결합이다.

	테이블 선택	few-shot	성능
C2: Pruning만	✓	✗	23.4%
C4: RAG	✓	✓	43.9%
few-shot 기여		+1	+20.5%p

RAG 전체 +22.4%p 중 **90%가 few-shot 예제**에서 온다.

발견 2 — Schema Pruning의 실체는 "짧은 컨텍스트"다

COMMENT가 없는 Pruned DDL의 기여는 +1.9%p로 작다.

하지만 Pruned DDL의 진짜 가치는 RAG 환경에서 나온다:

토큰 수를 3,635 → ~368로 줄여 **모델이 few-shot 예제에 집중**하게 한다.

발견 3 — Fine-tuning이 바꾸는 것은 "정확성"이다

base 모델은 SQL 구조는 알지만(유연 21%) 도메인 특화 조건을 모른다(정확 0.5%).

Fine-tuning이 정확 일치를 **82배** 향상시킨다.

RAG도 정확도를 27배 올리지만 Fine-tuning의 절반 수준.

발견 4 — Fine-tuned 모델에서 RAG는 오히려 역효과

조건	Blind 성능
v46 단독	83.2%
v46 + RAG	79.4%
차이	-3.8%p

v46은 "예제 없는 프롬프트" 형태로 학습했다 → RAG가 분포 이탈을 유발.

Fine-tuning 완료 후에는 RAG 없이 깔끔한 DDL만 사용하는 것이 최적.

발견 5 — Postprocessing과 Validator는 base에 효과 없음

- **Postprocessing**: base의 오류는 출력 포맷이 아닌 SQL 로직 자체의 문제
- **SQL Validator**: base도 SELECT-only 쿼리를 생성해 blocked=0 → 안전 차단 0건
- 두 기술 모두 **안전성·안정성** 목적이며, 순수 SQL 정확도 기여는 없다

8.5 inference-time 기술 운영 가이드

상황	권장 설정	기대 성능
base 모델만 있을 때	RAG (COMMENT 없이, Pruned DDL)	~44%
Fine-tuning 완료 후	깔끔한 DDL만 (RAG 불필요)	~83%
Fine-tuning + 추가 향상 원할 때	DDL COMMENT 추가 고려	~+5%p

9. 종합 결론

9.1 성능 요약

시기	모델	테스트셋	성능
3-table 초기	순수 base	blind1~4	~21%
3-table 완성	v21	blind1~4 평균	96.6%
3-table 완성	v21	blind5 (JOIN)	90.0%
6-table 초기	순수 base	Blind 214	21.5%
6-table 완성	v46	Blind 214 (raw)	83.2%
6-table 완성	v46	Blind 214 (TYPE_A 보정)	86.4%
6-table 완성	v46	Targeted 300	87.3%

9.2 기여도 서열

inference-time 기술만으로 달성 가능한 최고치: 43.9% (RAG 단독)
 Fine-tuning 단독: 83.2%

격차 : +39.3%p
 이 격차는 어떤 inference 기술로도 메울 수 없음
 격차의 85%는 Fine-tuning이 만든다

9.3 설계 원칙

이 프로젝트를 통해 확인한 NL2SQL 설계 원칙:

1. **Fine-tuning이 압도적이다**: 추론 기술 총합(~44%)보다 Fine-tuning 단독(83%)이 2배 강하다
2. **RAG는 few-shot 예제가 핵심이다**: 테이블 선택(+1.9%)보다 예제(+20.5%)가 10배 강하다
3. **Fine-tuning 후에는 RAG를 끄는 것이 낫다**: 학습 분포 이탈(-3.8%)
4. **Pruning의 가치는 컨텍스트 축소다**: 짧은 DDL이 모델의 집중력을 높인다
5. **평가 체계가 성능 못지않게 중요하다**: TYPE_A 보정 없이는 실제 성능을 3~7%p 과소평가한다

부록: 데이터 구축 이력

시점	훈련 데이터 수	주요 변경
3-table 초기 (v16)	866개	sc_users 단독
3-table 완성 (v21)	1,414개	sc_class, sc_user_orders JOIN 패턴 추가 + counter-example
6-table 전환 (v22~v38)	1,800~2,800개	sc_conf_* 3개 테이블 SQL 페어 대거 추가
데이터 정제 누적 (v39~v46)	~3,440개	IS NOT NULL 대조쌍, DDL COMMENT 반영, camelCase 패턴 보완
v46 최종	4,412개	IS NULL 대조쌍 완성, SELECT* → 컬럼 명시, 패턴 균형 보완

10. 향후 개선 방향

10.1 현재 남은 한계

v46 기준 여전히 해결되지 않은 문제들이다.

지표	현재 값	미달 원인
Blind raw	83.2%	36건 TYPE_C (진짜 오류) + 일부 flexible match 미처리
Coverage	75.9%	세 지표 중 최저 — 특정 컬럼/패턴 훈련 데이터 부족
v46 + RAG	-3.8%p	훈련이 "예제 없는 프롬프트" 형식으로만 진행 → RAG 분포 이탈

10.2 단기 개선 (즉시 실행 가능)

① TYPE_C 오류 → 훈련 데이터 추가

Blind 214개 중 36건 오답의 원인을 MySQL 실행으로 분류하면:

TYPE_C (진짜 오류)

→ 정답 SQL 페어로 변환 후 훈련 데이터 추가

EXEC_ERR (SQL 오류)

→ 컬럼명 환각 패턴 분석 후 DDL COMMENT 보강

BOTH_EMPTY (데이터 없음)

→ 평가 대상에서 제외 or 대표 쿼리 교체

오답 케이스를 직접 훈련 데이터로 편입하는 것이 가장 빠른 성능 향상 경로다.

② Coverage 약점 패턴 보강

Coverage 75.9%는 Blind 83.2%보다 7.3%p 낮다.

이는 특정 컬럼(특히 sc_conf_conference의 날짜 컬럼군, sc_conf_registrant의 결제 관련 컬럼)의 훈련 예제가 상대적으로 부족하다는 신호다.

coverage 테스트 결과

→ 컬럼별 정오답 분석 → 약점 컬럼 패턴 30~50개 추가

예상 효과: Coverage 75.9% → 80%+

③ flexible match 규칙 추가

현재 evaluate.py에서 동등하지만 다르게 처리되는 케이스:

```
-- 동일 의미지만 flexible match 미적용
NOT IN (SELECT ...) ↔ LEFT JOIN ... IS NULL

-- 추가 검토 필요
HAVING COUNT(*) > 0 ↔ EXISTS (subquery)
ORDER BY COUNT(*) DESC ↔ ORDER BY cnt DESC (alias)
```

이 규칙들을 추가하면 Blind 보정 점수가 추가 2~3%p 개선될 수 있다.

10.3 중기 개선 (기술적 전환 필요)

④ RAG-aware Fine-tuning (가장 유망한 방향)

현재 v46의 가장 큰 미해결 과제는 **Fine-tuning과 RAG가 함께 작동하지 못한다**는 점이다.

현재 구조:

v46 훈련 → "예제 없는 프롬프트"만 학습
추론 시 RAG 추가 → 학습 분포 이탈 → -3.8%p

목표 구조:

훈련 데이터 일부를 RAG 형식(예제 포함 프롬프트)으로 구성
→ 모델이 예제를 활용하는 법을 함께 학습
→ Fine-tuning 도메인 지식 + RAG in-context learning 시너지

기대 효과:

조건	현재	목표
Fine-tuning 단독	83.2%	83~85% (유지)
Fine-tuning + RAG	79.4% (-3.8%p)	88~90% (+5~7%p)

훈련 데이터를 혼합 구성하면 된다:

- **70%:** 기존 형식 (예제 없음) — 도메인 지식 학습
- **30%:** RAG 형식 (유사 예제 2~3개 포함) — 예제 활용 학습

⑤ Adaptive COMMENT 전략

현재 DDL COMMENT는 RAG 있을 때 오히려 -4.6%p 방해한다.

고정 ON/OFF 대신 **유사도 기반 동적 전환**을 적용한다.

```
# 개선 방안: 검색된 예제의 유사도가 낮으면 COMMENT로 보완
top_similarity = max([sim for _, sim in rag_results])
if top_similarity < 0.75:
    use_comment = True # 예제가 불충분 → COMMENT로 도메인 힌트 제공
else:
    use_comment = False # 예제가 충분 → COMMENT는 노이즈
```

⑥ 임베딩 기반 Schema Pruning 고도화

현재 Schema Pruning은 키워드 매칭 방식이다.

동일 임베딩 모델(paraphrase-multilingual-MiniLM)로 질문과 테이블 설명을 비교하면

"등록비가 얼마야?" → sc_conf_fee가 아닌 sc_user_orders를 잘못 선택하는 오류를 줄일 수 있다.

10.4 장기 연구 방향

⑦ 멀티턴 대화 지원

현재 시스템은 질문 하나에 SQL 하나를 반환하는 단발성 구조다.

```
현재: "2026년 등록자 수" → SELECT COUNT(*) ...
목표: "2026년 등록자 수" → SELECT COUNT(*) ...
후속: "이 중에서 정회원만" → 이전 컨텍스트 참조 → WHERE 조건 추가
```

피드백 플라이휠(feedback_store.json)과 결합하면

사용자가 수정한 SQL을 즉시 다음 쿼리의 예제로 활용할 수 있다.

⑧ SQL 실행 결과 기반 자동 검증

```
생성된 SQL 실행 → 결과 0건 → "조건이 너무 좁을 수 있음" 경고
생성된 SQL 실행 → 결과 10,000건 → "조건이 너무 넓음, LIMIT 확인" 경고
```

실행 결과를 모델 평가에도 반영하면 BOTH_EMPTY 케이스를 자동으로 걸러낼 수 있다.

⑨ 더 큰 모델로의 이전 (SQLCoder-34B)

현재 7B 모델의 이론적 성능 상한은 약 90% 내외로 추정된다.

RTX A6000 단일 GPU로는 34B QLoRA 훈련이 어렵지만,

어댑터 규모 유지 + 추론 전용으로 34B 모델 활용은 가능하다.

동일 훈련 데이터로 34B 기반 비교 실험을 통해 스케일 효과를 검증할 수 있다.

10.5 개선 우선순위

즉시 (1~2주):

- | | |
|-----------------------------|----------------------------|
| ① TYPE_C 오류 분석 + 훈련 데이터 추가 | → Blind +2~3%p 기대 |
| ② Coverage 약점 패턴 보강 | → Coverage 75.9% → 80%+ 기대 |
| ③ flexible match 규칙 2~3개 추가 | → 점수 인플레이션 없이 정당한 2%p 회복 |

중기 (1~2개월):

- | | |
|-------------------------|---------------------------------|
| ④ RAG-aware Fine-tuning | → Blind+RAG를 79.4%에서 88~90%로 반등 |
| ⑤ Adaptive COMMENT | → RAG+COMMENT 조합 손실 해소 |

장기 (2개월+):

- | | |
|----------------|---------------|
| ⑥ 멀티턴 대화 | → 사용성 대폭 개선 |
| ⑦ 34B 모델 비교 실험 | → 90%+ 가능성 검증 |

핵심 메시지: ④ RAG-aware Fine-tuning이 단일 개선 항목 중 기대 효과가 가장 크다.

현재 Fine-tuning(83.2%)과 RAG(43.9%)가 각자 강하지만 함께 작동하지 못하는 구조적 문제를 훈련 데이터 설계 변경만으로 해결할 수 있다.